

OGARD TECHNICAL PAPERS

Technical Whitepaper

Open Grid Asset Reliability Data Framework

Deniz Bektas

September 2026

OGARD Technical Whitepaper

Version 0.1.2

Deniz Bektas authored this whitepaper using original work, synthetic examples, and public sources.

Companion papers: OGARD Implementation Guide; OGARD Architecture and Data Model Specification; OGARD Synthetic Benchmark Methodology and Validation Plan.

Versioning: This paper is version 0.1.2. The methodology, model profile, and contradiction taxonomy it describes are version 0.1. The executable reference software has its own release numbers (Executable Reference Release v0.2.1 and later).

Repository: <https://github.com/DenBek/ogard>

Archive: <https://doi.org/10.5281/zenodo.22929401>

Documentation license: Creative Commons Attribution 4.0 International (CC BY 4.0).

Reference implementation license: Apache License 2.0 for original repository code; see LICENSE.md.

How to cite: Bektas, D. (2026). OGARD Technical Whitepaper: Open Grid Asset Reliability Data Framework. Version 0.1.2. <https://github.com/DenBek/ogard>

Abstract

OGARD is a reference methodology for integrating fragmented electricity-network asset records into auditable asset-evidence views and operator-governed derived status outputs. The framework addresses a recurring data-engineering problem: maintenance systems, asset registers, telemetry historians, fault and outage feeds, inspection records, and technical documents often describe the same physical equipment through different identifiers, time frames, and status semantics. OGARD separates the physical asset from the evidence records that describe it; models functional location, physical equipment identity, and telemetry identity as distinct but related concepts; preserves source provenance and contradiction; and routes uncertain mappings to explicit review. Its reference reconciliation design combines versioned deterministic rules, reproducible similarity scoring, and optional artificial-intelligence assistance that may recommend but may not become the decision authority. This whitepaper defines the framework, its governance boundary, its reference architecture, its release requirements, and a controlled approach to implementation and evaluation. The intended contribution is a reusable and inspectable engineering method that utility data teams can adapt inside their own controlled environments. Initial synthetic execution demonstrates selected reconciliation and review behavior; utility performance and operational outcomes are subjects for later implementation and evaluation.

Keywords: grid asset data; utility interoperability; entity resolution; data provenance; asset evidence; reconciliation; human review; data governance; auditability; electricity networks.

Author contribution

Deniz Bektas conceived the OGARD framework, authored this whitepaper, and developed the reference methodology. The project includes an initial software demonstrator with recorded execution across five synthetic scenarios.

Contents

Executive summary.....	1
1. Introduction and scope.....	2
2. The defined engineering problem.....	3
3. Relationship to existing standards and methods.....	4
4. Framework overview and design principles.....	6
5. Canonical asset-and-evidence model.....	7
6. Layered data architecture.....	9
7. Tiered reconciliation methodology.....	10
8. Contradiction handling and operator-governed status derivation.....	12
9. Governance, security, and production controls.....	13
10. Reference release and reproducibility.....	13
11. Implementation and evaluation approach.....	15
12. Limitations and threats to validity.....	16
13. Conclusion.....	17
References.....	18
Appendix A. Worked synthetic evidence example.....	19
Appendix B. Minimum mapping decision record.....	20
Appendix C. Glossary.....	21

Executive summary

Electricity-network operators rely on multiple information systems to plan, maintain, monitor, and respond to the condition of physical assets. These systems are commonly optimized for different purposes. A maintenance system may organize work around functional locations. An asset register may focus on installed equipment, ownership, or compliance. A historian may expose only telemetry tags. Fault feeds, inspections, and technical documents introduce additional identifiers and time frames. The same physical asset can therefore have several valid but incomplete representations.

OGARD addresses the engineering problem created when those representations cannot be reconciled reliably. It defines a reference method for preserving source evidence, resolving identity across systems, recording uncertainty, surfacing contradiction, and producing traceable information products for operator review. The method works alongside utility systems of record, IEC information models, enterprise asset-management products, historians, and operator procedures.

The framework has five core components:

1. A canonical asset-and-evidence model that separates physical assets from source records and preserves time, provenance, and review status.
2. A layered data architecture that retains raw evidence, makes conformance decisions explicit, and preserves lineage through to consumption outputs.
3. A tiered reconciliation workflow using deterministic rules first, reproducible similarity scoring second, and explicit human review where evidence remains insufficient or operational consequences are material.
4. A contradiction and derived-status workflow that preserves disagreement and produces operator-governed outputs rather than autonomous assertions of engineering truth.
5. Governance, security, observability, and change-control requirements designed for implementation within an operator-controlled environment.

OGARD's documentation comprises this whitepaper and companion papers on architecture and data modeling, implementation, and synthetic evaluation. The four papers are published in the OGARD GitHub repository (<https://github.com/DenBek/ogard>) and on the OGARD website (<https://denbek.github.io/ogard/>). An initial software demonstrator, version 0.1.0-demo.1, was executed across five selected synthetic scenarios and became the basis of the first public software release, 0.2.0. The demonstrator implements a bounded subset of the framework; broader synthetic evaluation, independent reproduction, and operator pilots remain development milestones.

OGARD is intended for utility data engineers, platform architects, asset-data specialists, data-governance teams, maintenance-technology teams, and practitioners implementing cross-system asset evidence. Its direct outputs are reconciled identities, preserved evidence, explicit contradictions, review queues, audit records, and operator-defined derived-status views. Any downstream reliability or restoration benefit depends on source quality, implementation discipline, operator action, and measurement in a real operating environment.

1. Introduction and scope

1.1. Operational context

Modern electricity systems are increasingly data-intensive. The NIST Smart Grid Interoperability Framework identifies information exchange and interoperability as foundational to grid modernization, particularly as the quantity and granularity of data increase [1]. At the same time, the North American Electric Reliability Corporation (NERC) projects substantial growth in North American summer and winter peak demand over the next decade [2]. The U.S. Department of Energy has also reported extended distribution-transformer lead times and supply constraints [3]. These conditions increase the practical value of understanding the installed asset base. OGARD addresses the associated engineering question: how can fragmented records about the same grid asset be reconciled and governed so that operators can inspect the evidence behind an information product?

1.2. Purpose

This whitepaper defines the conceptual and engineering foundation of OGARD. It provides a common vocabulary, identifies the trust boundaries of the method, and establishes the requirements that a reference implementation or operator pilot should satisfy. It supports technical inspection, practitioner feedback, and structured evaluation as the reference implementation develops.

How to use this revision. Read the technical method alongside the companion architecture and implementation materials. The initial demonstrator provides recorded examples of a bounded subset of the workflow; the Synthetic Benchmark Methodology defines the broader evaluation program. Each reported result should identify its artifact version, inputs, configuration, and evaluation method.

1.3. Intended audience

- Electricity-network data engineers and platform architects.
- Utility asset-management, maintenance-data, and reliability-technology teams.
- SCADA and historian data specialists working on tag and asset context.
- Data-governance, cybersecurity, and audit teams.
- Vendors and consultants implementing utility asset-data integration.
- Researchers and practitioners evaluating auditable entity-resolution methods.

1.4. Scope and claim boundaries

Use and governance boundary. OGARD is a technical reference methodology for operator-governed information products. Utility engineering, maintenance, cybersecurity, governance, and control-room procedures govern operational use. Formal engineering standards, safety assessment, certification, and regulatory approval remain separate processes.

Table 1. Scope and claim boundaries.

Area	OGARD v0.1 position
Primary scope	Electricity-network asset evidence and cross-system identity reconciliation.
Direct output	Auditable asset-evidence views, review decisions, contradictions, and operator-governed derived-status outputs.
Operator authority	Operators define status semantics, thresholds, decision rights, and acceptable use.
Artificial intelligence (AI)	Optional advisory support only. AI may propose or summarize but may not become the final authority.
Standards	Selected IEC Common Information Model (CIM) concepts provide mapping guidance; formal conformance and certification require separate assessment.
Validation status	Initial executable demonstration completed on five selected synthetic scenarios; broader benchmark evaluation and independent reproduction are planned.
Operational outcomes	Workflow and reliability outcomes require operator-based evaluation. Autonomous diagnosis and predictive maintenance are outside the demonstrated scope.
Adoption	Public documentation supports access and reuse. Operator adoption would be shown through separate implementation evidence.

2. The defined engineering problem

2.1. One physical asset, several information identities

A physical grid asset is not represented consistently across every system. The differences are not necessarily data errors. Each system models the asset for a particular business or operational purpose. The integration problem arises when a team needs to reason across those purposes and the systems do not share stable identifiers, synchronized time windows, or compatible status semantics.

Table 2. Five representations of one synthetic transformer.

Source class	Representation	Illustrative identifier or evidence
Maintenance system	Functional location and work history	FL-NE-SUB114-TX02; corrective work order WO-2026-0512; no equipment serial recorded
Asset register	Installed item or engineering record	AR-0047719; current unit serial TR-26-778204; register status Active from 15 April 2026
Telemetry historian	Observation channels	NE114.TX2.OILTMP; stale equipment reference to former unit serial TR-88-104553 after replacement
Technical documentation	Expected configuration and maintenance information	Current unit serial TR-26-778204; nameplate and maintenance guidance
Inspection or condition record	Condition evidence	Synthetic adverse observation associated with the current asset context

In this synthetic example, no single record proves the physical condition of the transformer. The register, maintenance record, telemetry feed, and inspection result answer different questions. The correct response is not to choose one source silently. The method should preserve the evidence, establish which records refer to the same physical context, identify the disagreement, and route the resulting information to the appropriate operator process.

2.2. Three identity regimes

OGARD treats grid-asset identity as three related regimes rather than one generic asset key: functional location, physical equipment identity, and telemetry or instrumentation identity. These concepts can have different life cycles. A functional location may persist through equipment replacement. A serial-numbered unit may move. A telemetry tag may be renamed, re-pointed, duplicated, or left stale. Many apparent cross-system inconsistencies are caused by treating these regimes as interchangeable.

2.3. Evidence fragmentation

Evidence relevant to one asset may be distributed across work orders, fault events, telemetry, planned outages, inspection results, technical documents, and environmental or operating context. Records may arrive late, refer to historical equipment, use local naming conventions, or disagree on status. A conventional integration that produces a single golden record can conceal these differences if it overwrites losing values or fails to preserve the decision path.

2.4. Consequences for data and review workflows

- Manual reconstruction of asset identity and history across several systems.
- Repeated effort to resolve the same naming and replacement patterns.
- Inability to explain why a mapping or derived status was accepted.
- Hidden contradictions between maintenance, register, telemetry, and inspection evidence.
- Overconfidence when incomplete or stale records are collapsed into one status.
- Difficulty comparing results across teams because rules and thresholds are undocumented.

The framework treats uncertainty as an explicit output to be managed through review and documented decisions.

3. Relationship to existing standards and methods

3.1. NIST interoperability framework

The NIST Smart Grid Interoperability Framework provides a broad architecture and standards roadmap for information exchange across the grid [1]. OGARD is consistent with that interoperability objective but operates at a more specific implementation layer: preserving and reconciling evidence about physical assets across heterogeneous records.

3.2. IEC Common Information Model

IEC 61970-301 defines the Common Information Model as an abstract model for major objects involved in utility operations and supports a common semantic language for integration [4]. IEC 61968-11 provides distribution-related CIM extensions [5]. OGARD uses selected CIM concepts as mapping guidance for its canonical model. CIM answers an important semantic question: how can utility information be represented in a shared model? OGARD focuses on a complementary implementation question: how should inconsistent source records be mapped into a governed representation while preserving evidence, uncertainty, and review decisions?

This revision uses selected CIM concepts as mapping guidance. A formal CIM profile, complete crosswalk, and conformance test would be separate implementation deliverables.

3.3. Provenance models

The W3C PROV family provides a general model for representing entities, activities, agents, and derivation relationships across heterogeneous contexts [6]. OGARD uses the principle that a result should be traceable to the records and activities that produced it. A formal mapping to PROV-O is a separate implementation task.

3.4. Entity resolution and record linkage

Record linkage and entity resolution are mature technical fields. Classical probabilistic linkage formalized the comparison of records under uncertainty [7]. Later work describes deterministic, probabilistic, similarity-based, and learned methods for data matching [8, 9]. OGARD's contribution is the disciplined composition of established matching methods with grid-specific identity regimes, time validity, contradiction preservation, human review, and audit requirements.

3.5. Machine-assisted schema and record matching

Recent research indicates that large language models can assist schema matching and related semantic tasks, but results remain dependent on context and require verification [10]. OGARD therefore treats any language-model output as a recommendation. The method may use a model to rank candidates, classify evidence, or draft an explanation, but a probabilistic model must not silently alter a canonical asset record or determine an operational status.

3.6. The contribution boundary

OGARD's contribution is a documented and inspectable method that combines CIM-informed modeling, provenance, entity resolution, data lineage, human review, and governed machine assistance to address a defined grid-asset evidence problem.

3.7. Comparative positioning

Master data management (MDM) platforms, CIM harmonization programs, and general-purpose entity-resolution libraries provide valuable storage, semantic, lineage, matching, and survivorship capabilities. OGARD builds on these capabilities to address a specific implementation problem: reconciling grid-asset evidence while distinguishing functional location, installed equipment, and telemetry identity.

Compared with conventional MDM, OGARD does not define success as producing one surviving golden value. It requires the disagreement history, time-valid installation relationships, mapping decisions, and unresolved contradictions to remain inspectable. Compared with CIM harmonization, it focuses on how inconsistent source evidence is admitted, mapped, reviewed, and governed rather than claiming a complete semantic profile. Compared with general entity-resolution tools, it adds grid-specific identity regimes, operator-controlled decision authority, contradiction as a first-class output, and an explicit safe Unknown state.

OGARD provides a compositional, domain-specific control pattern for applying established integration and matching techniques while preserving evidence, temporal identity, uncertainty, and operator authority for electricity-network assets.

4. Framework overview and design principles

Table 3. OGARD framework components.

Component	Purpose	Primary output
Canonical asset-and-evidence model	Separate the physical asset from the records that support or challenge its representation.	Canonical identities, evidence links, time validity, status provenance.
Layered data architecture	Preserve raw evidence and make normalization, mapping, and consumption decisions explicit.	Traceable records from source through consumption.
Tiered reconciliation workflow	Resolve cross-system identity using controlled rules, scores, and review.	Mapping decisions recorded as accepted, rejected, review required, deferred, escalated, unresolved, or contradiction.
Contradiction and derived-status workflow	Preserve disagreement and apply operator-approved status rules.	Contradictions, review queues, and derived-status outputs.
Governance and production controls	Define decision rights, security, change control, monitoring, and audit.	Operable and reviewable implementation controls.

4.1. Evidence preservation

Source records should be retained with sufficient fidelity to reconstruct the input to every material transformation. Conformance and canonical outputs should not erase the original values. When a source value loses a survivorship decision, it remains available as evidence.

4.2. Explicit uncertainty

The method should expose insufficient, stale, ambiguous, or contradictory evidence. An Unknown or review-required result is preferable to a false precise answer. Review queues and contradiction records are first-class outputs, not implementation failures.

4.3. Auditability

Every accepted mapping, rejected candidate, threshold decision, review action, and derived status should be explainable from stored records. The relevant rule or model version, score, reviewer action, and source evidence should be available to an authorized reviewer.

4.4. Operator authority

OGARD provides a reference method. The operator remains responsible for asset taxonomies, status definitions, thresholds, review roles, security controls, and decisions that affect operating practice. A public reference implementation cannot define operational truth for an operator it does not know.

4.5. Platform neutrality

The framework is expressed through data contracts, schemas, rules, review records, and governance requirements rather than a dependency on one software vendor. Implementations may use different databases, data platforms, workflow tools, and visualization systems, provided the stated guarantees are preserved.

4.6. Minimum necessary use of artificial intelligence

Artificial intelligence should be used only where it adds a clear, measurable benefit and where its failure modes can be bounded. Deterministic rules should be preferred for stable, well-understood relationships. Human review should remain available when evidence is ambiguous or consequences are material.

5. Canonical asset-and-evidence model

5.1. Core separation

The central modeling decision is to separate the physical or operational asset context from the source records that describe it. A canonical asset is a governed representation. An evidence record is a preserved source observation, document, or transaction. A canonical representation may change as evidence is added or corrected. The evidence history should remain traceable.

5.2. Identifier regimes

Table 4. Grid-asset identifier regimes.

Regime	Meaning	Common risk
Functional location	The role or position occupied in the network or operating hierarchy.	Persists through replacement; may change under network reconfiguration.
Physical equipment identity	The installed item, commonly associated with serial, manufacturer, or nameplate information.	May move, be replaced, or be recorded incompletely.
Telemetry or instrumentation identity	The channels through which behavior is observed.	May be renamed, reused, re-pointed, duplicated, or stale.

5.3. Core entities

Table 5. Core canonical and evidence entities.

Entity	Role	Minimum information
CanonicalAsset	Stable governed reference context for an asset within the selected scope.	asset_id, asset_class, operator scope, created_at, updated_at, model_profile_version; schema_version
FunctionalLocation	Role or position in the network hierarchy.	functional_location_id, parent or hierarchy, location type, valid_from, valid_to, model_profile_version; schema_version
EquipmentUnit	Physical installed item.	equipment_unit_id, serial or alternative identity, equipment class, manufacturer/model where available, lifecycle dates, model_profile_version; schema_version
AssetInstallation	Time-valid relationship between a functional location and equipment unit.	installation_id, functional_location_id, equipment_unit_id, valid_from, valid_to, evidence reference, model_profile_version; schema_version
TelemetryChannel	Observation channel associated with a location, item, or aggregate.	channel_id, source system, tag, measurement type, unit, valid_from, valid_to, mapping state, model_profile_version; schema_version

Entity	Role	Minimum information
EvidenceRecord	Preserved source record or controlled source reference.	evidence_record_id, source system, source record ID, record type, timestamps, batch or stream identifier, security classification, schema version, payload reference, quality state, model_profile_version
EvidenceRecord subtype examples	Typed evidence for maintenance, faults or outages, inspections or condition records, and technical documents.	evidence_record_id, record_type, subtype-specific fields, source references, model_profile_version; schema_version
MappingDecision	Auditable disposition of a candidate relationship.	decision_id, source record, candidate context, method tier, rule or model identifier and version, score or confidence, threshold configuration, outcome, actor, reason code, timestamps, effective validity period, evidence references, model_profile_version; schema_version
ReviewAction	Authorized human disposition of a review item.	review_action_id, review item, reviewer, decision, reason code, timestamp, evidence references, model_profile_version; schema_version
Contradiction	Typed disagreement or unresolved conflict between evidence classes.	contradiction_id, type, severity, implicated context, triggering evidence, state, owner, resolution, model_profile_version; schema_version
DerivedAssetStatus	Operator-governed status with evidence, rule, time, and review provenance.	status_id, asset_id, status, effective interval, evidence set, rule version, review state, contradiction references, model_profile_version; schema_version

5.4. Time validity

Identity and status are time-dependent. Equipment can be replaced at a stable functional location. Telemetry tags can be re-pointed. Work orders can arrive after the equipment context changes. OGARD therefore requires effective intervals or equivalent temporal controls for mappings that may change over time. A mapping that is valid today should not be applied automatically to a historical record without checking its effective period.

5.5. Provenance and evidence sets

A derived output should reference the evidence set, transformation or rule version, review state, and relevant timestamps that support it. Provenance should be sufficiently detailed to answer: Which source records were used? Which candidates were considered? Which rule or score caused the route? Who accepted or rejected the result? What changed from the prior version?

Versioning requirement. Every persisted canonical, evidence, mapping, review, contradiction, derived-status, and audit record must carry, or be traceable to, both model_profile_version and schema_version. Historical outputs must remain traceable to the model and schema in force when the decision was produced; later model changes must not silently reinterpret prior records.

5.6. Reference derived-status taxonomy

OGARD proposes a reference vocabulary, not a universal operational classification. Each operator must define the evidence and authority required for each status. The status should be reported with confidence or review state and should not be treated as an autonomous engineering determination.

Table 6. Reference derived-status taxonomy.

Status	Reference meaning	Minimum interpretation safeguard
Available	Approved evidence supports expected availability within the defined scope, and no unresolved material contradiction blocks the classification.	Availability is assessed within the defined evidence scope. The operator defines coverage, freshness, and disqualifying contradictions; latent defects require separate assessment.
Degraded	Approved evidence supports impaired condition or reduced capability under an operator-defined rule.	Requires asset-class-specific criteria, time validity, and authorized engineering interpretation.
Offline	Approved evidence supports that the asset is unavailable, disconnected, or out of service.	Must distinguish planned, forced, temporary, and data-quality conditions using time-valid evidence.
At risk	Approved evidence indicates a condition requiring attention but does not support Degraded or Offline.	Must not be represented as a failure prediction; indicators, expiry, and review requirements must be defined.
Unknown	Evidence is insufficient, stale, ambiguous, outside scope, or materially contradictory.	Unknown is the safe default whenever another status cannot be justified.

6. Layered data architecture

The reference architecture contains four logical processing layers: raw evidence, conformance, canonical asset-and-evidence, and consumption. A source-interface boundary governs approved ingestion, while review, contradiction, and audit functions operate across the canonical and consumption stages. A technology implementation may combine or subdivide physical components, but these logical responsibilities should remain distinct.

Logical flow: approved source interface – raw evidence – conformance – canonical asset-and-evidence – controlled consumption, with review, contradiction, and audit functions applied where required.

Table 7. Layered architecture and guarantees.

Logical layer	Responsibilities	Guarantee
Raw evidence layer	Preserve original values or payload references, source identifiers, extraction time, ingestion time, and schema version.	The source evidence required for audit remains available under approved retention rules.
Conformance layer	Apply schema checks, parsing, normalization, time standardization, duplicate handling, and quarantine decisions.	Records become comparable and their quality state is explicit without erasing source evidence.
Canonical asset-and-evidence layer	Resolve identity, maintain time-valid relationships, link evidence, record mapping and review decisions, and preserve contradictions.	Every material relationship and derived output remains traceable to evidence, rules, and review records.
Consumption layer	Expose authorized asset-evidence views, derived-status views, queues, and measurement outputs.	Consumers receive governed outputs with lineage, review state, and use restrictions.

6.1. Source contracts

Each source should have a documented contract covering schema, delivery method, refresh expectations, primary identifiers, timestamps, quality checks, classification, and intended use. A source should not be

treated as authoritative merely because it is labeled a system of record; authority may differ by attribute and time period.

6.2. Raw evidence preservation

The raw layer should be append-oriented where practicable. Corrections and reprocessing should occur through versioned downstream transformations rather than silent modification of historical source evidence. Sensitive payloads may be referenced rather than copied if security or retention rules require that design.

6.3. Conformance and quality state

Conformance should standardize representation without erasing meaning. Unit conversion, identifier parsing, timestamp normalization, and duplicate handling should record the original value, the transformation applied, and any quality or quarantine status. Late and out-of-order records require explicit treatment.

6.4. Consumption boundaries

Consumption outputs should be role-based. A general data consumer may see a summarized status and provenance reference, while an authorized reviewer may inspect full source evidence and decision history. Sensitive operating data should remain within the operator-controlled environment and should not be exported into a public reference repository.

7. Tiered reconciliation methodology

Reconciliation is an auditable decision workflow, not a single algorithm. It begins with candidate generation, applies controlled methods in order of determinism, and routes unresolved or consequential cases to review. Every route should be reproducible from a named configuration.

Table 8. Reconciliation tiers and decision authority.

Stage	Method	Permitted outcome
Candidate generation	Region, hierarchy, asset class, identifier tokens, time windows, and other domain constraints.	Produce a bounded candidate set. No final mapping.
Tier 1	Exact relationships, approved aliases, deterministic rules, and explicit source links.	Automatic acceptance only under a versioned rule with tests and approved scope.
Tier 2	Reproducible similarity measures and constrained re-scoring.	Accept, reject, or review according to documented thresholds and risk class.
Tier 3	Optional machine-assisted recommendation or evidence summary.	Recommendation only. No direct canonical mutation or final status determination.
Human review	Authorized comparison of candidates, evidence, scores, and time context.	Accept, reject, defer, escalate, mark contradiction, or propose a new rule.

7.1. Candidate generation and blocking

Candidate generation reduces the comparison space using known constraints such as region, substation, equipment class, identifier fragments, installation windows, document references, and telemetry naming conventions. Blocking rules should be versioned and tested because an overly restrictive rule can exclude the true match before scoring begins.

7.2. Deterministic rules

Deterministic rules are appropriate where relationships are stable and explainable. Examples include an approved alias table, an exact serial number under an effective installation interval, an explicit work-order asset link, or a documented telemetry naming convention. Each rule should have an identifier, version, owner, scope, effective date, test cases, and decision authority.

7.3. Similarity-based scoring

Similarity methods may help when deterministic evidence is incomplete. Techniques can include token-set similarity, token-sort similarity, Jaro–Winkler similarity, character n-gram similarity, and constrained embedding re-scoring. No method should be treated as universally superior. Thresholds must be calibrated by source quality, asset class, and consequence. A high score is evidence, not proof.

7.4. Human review

A review item should present the candidate source record, candidate asset contexts, relevant identifiers, time windows, scores, rules applied, contradictions, and available supporting evidence. The reviewer should select from controlled outcomes and record a reason code. Review ownership must be defined before an implementation can be considered governed.

7.5. Optional artificial-intelligence assistance

A language model or other probabilistic component may assist with candidate explanation, document classification, field mapping, evidence summarization, or queue prioritization. The output should include the task version, model identity where available, source references, confidence or uncertainty indicator, and disposition. It should not create a final mapping, alter a canonical asset record, determine a derived status, or determine fault cause without explicit authorized acceptance.

7.6. Feedback into the rule library

Repeated, well-supported review decisions may justify a new deterministic rule. Promotion should not occur automatically. The proposed rule should be documented, tested against historical examples and counterexamples, approved by an authorized owner, and monitored after deployment. The rule version should remain linked to every decision it produces.

7.7. Worked reconciliation example

A synthetic corrective work order references functional location FL-NE-SUB114-TX02 and contains the description “SUB 114 TRANSF 2 OIL LEAK” but no serial number. A deterministic rule maps the functional location to a canonical location. Installation history shows that two equipment units occupied the location during adjacent periods. Similarity scoring ranks the newer unit higher, but the result falls below the auto-accept threshold. (This step shows the reference design; the executed demonstrator routes this work order to review because it carries no equipment identifier.) The record is routed to review. A synthetic authorized reviewer confirms the newer unit using the work-order date and installation interval. The accepted decision is logged, and the date-window pattern is proposed for a future deterministic rule. At no point is the ambiguous equipment link accepted silently.

8. Contradiction handling and operator-governed status derivation

8.1. Contradiction as information

A contradiction is a typed disagreement between evidence classes or time contexts. It should not be deleted merely because one source wins a survivorship rule. The contradiction may indicate stale configuration, incomplete replacement history, tag drift, delayed maintenance closure, or another issue requiring review.

Table 9. Illustrative contradiction categories.

Category	Illustrative condition	Typical route
Register versus maintenance	Register shows active while a material corrective work order remains open.	Asset-management or maintenance review.
Register versus telemetry	Register shows active while expected telemetry has been absent beyond an approved interval.	SCADA or historian review plus data-quality assessment.
Serial versus location	One serial appears at two locations during overlapping intervals.	Installation-history and asset-register review.
Telemetry tag drift	A tag continues to report after the mapped equipment was replaced.	Instrumentation mapping review.
Document versus installation	Applicable technical document conflicts with recorded installed configuration.	Engineering document and configuration review.
Event versus status	Fault or outage evidence conflicts with a current available status.	Operational event review and status re-evaluation.

8.2. Rule-based derived status

A derived status should be produced only under an operator-approved rule that defines the required evidence, exclusions, time windows, contradiction conditions, and decision authority. The output should include its effective time, supporting evidence, rule version, and review state. Where a material contradiction remains unresolved, the result may be Unknown or review required.

8.3. Diagnostic-support information

OGARD may assemble a structured evidence package for operator-led investigation. Such a package can include recent telemetry, work and fault history, inspection records, configuration context, technical-document references, and unresolved contradictions. The framework does not determine fault cause. It organizes and prioritizes evidence so that authorized practitioners can investigate more efficiently.

8.4. Priority queues

A review or exception queue may consider operator-provided asset criticality, contradiction severity, time in state, recurrence, evidence freshness, and known deadlines. Every ranking factor should be explainable. A queue rank is a workflow priority, not a prediction of failure.

9. Governance, security, and production controls

9.1. Decision rights

An implementation should identify who owns the source, who approves an identifier mapping, who resolves contradictions, who defines derived-status rules, who can view sensitive evidence, and who can authorize production use. Technical capability without decision-right clarity is not a governed implementation.

9.2. Cybersecurity and data handling

The NIST Cybersecurity Framework 2.0 provides a high-level taxonomy for managing cybersecurity risk across organizations [11]. OGARD does not prescribe a complete cybersecurity program, but an operator implementation should align with the operator's approved risk-management framework and include least-privilege access, identity and access management, encryption, secrets management, logging, retention controls, secure development practices, and incident response.

9.3. Change control

Schemas, source mappings, rules, thresholds, models, prompts, measurement definitions, and status semantics should be versioned. Material changes should have an owner, rationale, testing record, approval, deployment date, and rollback path. A historical decision must remain traceable to the configuration that was in force when the decision occurred.

9.4. Testing and observability

Testing should cover schema behavior, rules, edge cases, time windows, duplicate handling, negative examples, and regression behavior. Operational monitoring should cover data freshness, source latency, pipeline failure, quarantine volume, mapping rates, review-queue depth, contradiction volume, audit-log completeness, and availability of controlled outputs.

9.5. Data retention and confidentiality

Retention should follow legal, regulatory, contractual, security, and operational requirements established by the operator. Public OGARD artifacts should contain only original, synthetic, or public data. Sensitive source records, credentials, internal configurations, and operator-specific implementation detail should not be copied into public documentation or repositories.

9.6. Production readiness boundary

Production use requires source ownership, security approval, review ownership, monitoring, rollback, incident response, retention rules, data-quality controls, and operator acceptance. Reference implementations and synthetic evaluations support preparation for those deployment controls.

10. Reference release and reproducibility

10.1. Documentation and demonstrator status

The four papers are published in the OGARD GitHub repository (<https://github.com/DenBek/ogard>) and on the OGARD website (<https://denbek.github.io/ogard/>). This revised whitepaper incorporates the initial demonstrator's documented execution. Paper publication, demonstrator execution, software distribution,

and any independent evaluation are tracked separately, each against its own version and supporting record. Version-controlled releases, release notes, explicit licenses, and archival snapshots support future reuse and verification.

10.2. Documentation and software artifact inventory

Table 10. Documentation, demonstrator evidence, and further release artifacts.

Artifact	Function or release objective	Documented status in this revision
Technical whitepaper	Defines scope, method, boundaries, and references.	This paper; published with the three companion papers.
Implementation guide	Defines readiness, roles, implementation sequence, and controls.	Companion paper, published separately in the same release.
Architecture and data-model specification	Provides original diagrams, entity definitions, trust boundaries, and minimum conformance requirements.	Companion paper, published separately in the same release.
Synthetic benchmark methodology and validation plan	Defines evaluation questions, ground-truth separation, baselines, metrics, lifecycle accounting, reproducibility, and the results-release gate.	Companion paper, published separately in the same release; defines broader evaluation beyond the initial five-scenario demonstration.
Canonical schema example	Machine-readable original example with version and documentation.	Run manifest identifies machine-readable example files; formal schema-package coverage remains a development item.
Synthetic data or generator	Original synthetic records and known ground truth.	Run manifest identifies data/cases.json; execution records cover five selected synthetic scenarios. Broader datasets and generation remain planned.
Rule and scoring configuration	Versioned deterministic rules, similarity methods, and thresholds.	Manifest identifies demonstrator engine and illustrative-rule artifacts; broader rule and similarity-scoring coverage remains planned.
Evaluation logic	Recomputes any published metrics from the released artifacts.	Manifest identifies evaluator code; evaluation.json records case-level outputs and checks for the initial run.
Expected outputs and audit sample	Allows comparison with a named release configuration.	decisions.json and evaluation.json document initial outputs; test results and an internal rerun record accompany the run.
README and changelog	Explains installation, use, limitations, version history, and known issues.	README.md and review/provenance files are identified in the run manifest. Public software-release documentation is tracked separately.
Independent review records	Identifies the version reviewed, scope, findings, limitations, and author response.	Reproduction records under the benchmark methodology are a development item. The recorded rerun was internal.

10.3. Reproducibility standard

When a release reports benchmark results, a technically competent practitioner who did not author the artifacts should be able to obtain the tagged release, follow the documented environment and commands, generate or load the synthetic estate, run the reconciliation and evaluation logic, and compare the result with

the expected output. Failures and discrepancies should be recorded as defects or known limitations rather than suppressed.

10.4. Benchmark boundary

The initial demonstrator covers five selected synthetic scenarios, with execution records maintained for version 0.1.0-demo.1. It provides concrete examples of implemented reconciliation and review behavior. A broader benchmark will evaluate the framework across predefined datasets, baselines, and metrics, supported by versioned inputs, code, configuration, expected outputs, and documented reproduction.

The initial demonstrator's recorded run (0.1.0-demo.1) included 15 passing software tests and an internal rerun that reproduced its saved outputs. Executable Reference Release v0.2.1 runs 33 automated tests, all passing. Its reviewer actions are scripted. One scenario records an incorrect automatic association when sources repeat the same wrong equipment identifier. Passing tests confirm the stated software checks, including the evaluator's identification of that error against hidden synthetic truth; they do not mean every association was correct. Similarity scoring, optional runtime AI assistance, and the full derived-status workflow remain reference-design capabilities beyond this initial demonstrator. Executable Reference Release v0.2.1 presents the five scenarios, six decisions, and supporting records at <https://github.com/DenBek/ogard/tree/v0.2.1/demonstrator/results>.

10.5. Independent review

Independent review should identify the reviewer, relevant expertise, relationship to the author, exact version reviewed, materials examined, review scope, findings, and limitations. Further evaluation should include technical architecture review and utility-domain relevance review. Before broader benchmark results are represented as independently reproduced, a separate clean-environment reproduction attempt should be documented. Public review records should identify their technical assessment scope and any specific findings.

10.6. Versioning and change history

Each public release should have a unique version, date, tagged repository state, release notes, and immutable or archived copy where possible. Corrections that alter technical meaning should create a new release or revision. The website may describe the latest state, but the archived release should remain the version of record for that version.

11. Implementation and evaluation approach

11.1. Readiness assessment

A pilot should begin only after the operator identifies an asset class, source-system owners, data-access methods, security classification, review roles, and a bounded question. Each source should be assessed for identifier coverage, timestamp quality, history, refresh frequency, ownership, extraction method, known quality issues, and intended use.

11.2. Bounded pilot

A practical pilot may focus on one asset class and two to four source classes. It should be large enough to expose identity and evidence problems but small enough for authorized subject-matter experts to review

outcomes. The pilot should define entrance criteria, baseline measures, success conditions, stop conditions, and the evidence that may be reported externally.

11.3. Suggested implementation sequence

1. Confirm scope, decision rights, security boundaries, and measurement definitions.
2. Inventory sources, identifiers, time fields, evidence classes, and known data-quality issues.
3. Build raw preservation and conformance controls with lineage and quarantine behavior.
4. Implement the minimum canonical and evidence model for the selected asset class.
5. Configure deterministic rules, similarity methods, thresholds, and review routing.
6. Implement contradiction handling and operator-approved derived-status rules.
7. Run the pilot, work the review queue, measure results, and record unresolved issues.
8. Assess production readiness and decide whether to stop, revise, expand, or proceed.

11.4. Measurement classes

Table 11. Measurement classes and evidentiary boundaries.

Class	Examples	What it can support
Public release metrics	Artifact completeness, documentation coverage, issue status, release history.	Evidence that the framework is documented and disseminated.
Synthetic benchmark metrics	Precision, recall, routing share, unresolved review volume, contradiction detection, reproducibility.	Behavior against known synthetic ground truth only.
Operator pilot activity metrics	Source coverage, asset coverage, review volume, audit completeness, rule coverage.	Implementation activity within the defined pilot scope.
Operator outcome metrics	Time to assemble evidence, contradiction-resolution time, status coverage, reviewer throughput.	Operational workflow change only when measured against an approved baseline.
Reliability outcomes	Outage duration, restoration measures, asset failures, maintenance outcomes.	Require operator records, causal caution, adequate time period, and separate validation.

11.5. Evidence and claims discipline

Each reported result should identify its evidence class, data type, period, scope, method, exclusions, and limitations. Synthetic measures describe behavior against controlled inputs; pilot measures describe implementation and workflow change; reliability outcomes require separate operator records and evaluation. Keeping these classes distinct makes progress and its practical implications assessable.

12. Limitations and threats to validity

12.1. Operator validation

The initial execution evidence concerns synthetic scenarios. Live operator evaluation will need to assess local data, controls, workflows, and operating consequences, which can differ materially from synthetic examples. Production suitability requires evaluation within that operating context.

12.2. Synthetic-data limitations

Synthetic data enables known ground truth and controlled testing, but it cannot reproduce the full irregularity, history, access constraints, or organizational context of a live estate. A generator can also encode assumptions that favor the method being evaluated. Benchmark design should therefore include holdout cases, negative examples, sensitivity analysis, and independent reproduction.

12.3. Domain coverage

The reference model defines a focused grid-asset evidence profile. Asset classes, condition evidence, regulatory requirements, and operating semantics vary. The framework should be extended only with qualified domain input and should not be treated as suitable for every asset class by default.

12.4. Standards mapping limitations

Formal CIM conformance requires a complete profile, version-specific crosswalk, serialization rules, and conformance testing. This revision provides selected mapping guidance.

12.5. Source quality and authority

Source ownership, quality, and authority are implementation constraints. Inaccessible, stale, or poorly understood sources require explicit quality handling, and insufficient evidence should yield an Unknown result.

12.6. Organizational and human factors

Review capacity, role clarity, local naming knowledge, maintenance practice, and incentives can determine whether the method is usable. A technically correct queue that no authorized person owns is not an effective control.

12.7. Artificial-intelligence limitations

Model behavior can change with prompts, versions, context, and source quality. A model can produce plausible but unsupported explanations. OGARD therefore does not depend on artificial intelligence for its core guarantees, and any optional use requires separate evaluation, provenance, and review.

12.8. Reliability outcomes

Better asset evidence can support operator decisions. Establishing effects on failures, outages, restoration time, or cost requires evaluation of implementation, operator action, system conditions, and appropriate comparison periods. These are downstream objectives for operator-based evaluation.

13. Conclusion

OGARD defines a reference method for reconciling fragmented grid-asset evidence without hiding uncertainty or transferring decision authority away from the operator. The method separates assets from evidence, models distinct identity regimes, preserves provenance and contradiction, routes uncertain mappings to review, and requires versioned rules and audit records. Its technical value should be assessed through inspectable artifacts, independent review, reproducible synthetic evaluation, and carefully bounded operator pilots.

OGARD now combines publicly available technical documentation with an initial executable demonstration of its reconciliation and review approach. The next development stage is to expand the implemented coverage, test more varied asset and evidence conditions, support independent reproduction under the benchmark methodology, and assess practical use through bounded operator pilots. Versioned artifacts and transparent results will allow practitioners to inspect, adapt, and evaluate the method as it develops.

References

- [1] National Institute of Standards and Technology. NIST Framework and Roadmap for Smart Grid Interoperability Standards, Release 4.0. NIST Special Publication 1108r4, 2021. <https://doi.org/10.6028/NIST.SP.1108r4>
- [2] North American Electric Reliability Corporation. 2025 Long-Term Reliability Assessment. January 2026. https://www.nerc.com/globalassets/our-work/assessments/nerc_ltra_2025.pdf
- [3] U.S. Department of Energy, Office of Electricity. Distribution Transformers. Accessed September 2026. <https://www.energy.gov/oe/distribution-transformers>
- [4] International Electrotechnical Commission. IEC 61970-301:2020+AMD1:2022, Energy management system application program interface (EMS-API) – Part 301: Common information model (CIM) base. <https://webstore.iec.ch/en/publication/74467>
- [5] International Electrotechnical Commission. IEC 61968-11:2013, Application integration at electric utilities – System interfaces for distribution management – Part 11: Common information model (CIM) extensions for distribution. <https://webstore.iec.ch/en/publication/6199>
- [6] W3C Provenance Working Group. PROV-O: The PROV Ontology. W3C Recommendation, 30 April 2013. <https://www.w3.org/TR/prov-o/>
- [7] Fellegi, I. P., and Sunter, A. B. A Theory for Record Linkage. *Journal of the American Statistical Association*, 64(328), 1183–1210, 1969. <https://doi.org/10.1080/01621459.1969.10501049>
- [8] Christen, P. *Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection*. Springer, 2012. <https://doi.org/10.1007/978-3-642-31164-2>
- [9] Winkler, W. E. Overview of Record Linkage and Current Research Directions. U.S. Census Bureau Research Report Series, RRS2006-02, 2006. <https://www.census.gov/library/working-papers/2006/adrm/rrs2006-02.html>
- [10] Parciak, M., Vandevoort, B., Neven, F., Peeters, L. M., and Vansummeren, S. Schema Matching with Large Language Models: An Experimental Study. arXiv:2407.11852, 2024. <https://doi.org/10.48550/arXiv.2407.11852>
- [11] National Institute of Standards and Technology. The NIST Cybersecurity Framework (CSF) 2.0. NIST Cybersecurity White Paper 29, 2024. <https://doi.org/10.6028/NIST.CSWP.29>

Appendix A. Worked synthetic evidence example

This appendix illustrates the intended decision record using synthetic equipment and evidence. The worked method includes proposed similarity and derived-status behavior; the separate initial demonstrator implements and records a bounded subset of that workflow.

A.1. Input evidence

- Maintenance record: corrective work order WO-2026-0512 at functional location FL-NE-SUB114-TX02, opened after a documented equipment replacement and containing no equipment serial number.
- Asset-register record: current equipment unit TR-26-778204 recorded as active at the location from 15 April 2026.
- Installation-history record: former equipment unit TR-88-104553 recorded at the location until 14 April 2026 and replacement unit TR-26-778204 recorded from 15 April 2026.
- Telemetry record: tag NE114.TX2.OILTMP remains explicitly associated with former equipment unit TR-88-104553 after the replacement date.
- No live condition determination is made; the example tests identity, review, contradiction, status limitation, and audit behavior only.

A.2. Decision path

1. The functional location is resolved deterministically under an approved alias or direct relationship rule.
2. Two historical equipment candidates are generated because both occupied the location at different times.
3. The work-order timestamp and installation intervals rank the current equipment unit above the removed unit, but the available evidence remains below the declared auto-accept threshold.
4. The candidate set, temporal evidence, and score are routed to an authorized reviewer, who confirms the current equipment unit and records the reason and evidence references.
5. The telemetry channel remains explicitly associated with the former equipment unit after the replacement date. That stale tag-to-equipment relationship is preserved as a contradiction and requires authorized telemetry-owner resolution.
6. The accepted equipment mapping is recorded separately from the unresolved telemetry contradiction; neither silently overwrites the other.
7. Any disagreement between register status, work-order state, telemetry freshness, and inspection evidence is stored as a contradiction. A status that depends on unresolved telemetry or contradictory evidence remains Unknown or carries an explicit limitation under operator rules.

A.3. Result

The output is a governed evidence view containing the reviewer-approved identity relationship, the unresolved telemetry relationship, contradiction records, an Unknown or explicitly limited status where required, and the evidence needed for an authorized operator process.

Appendix B. Minimum mapping decision record

Table 12. Minimum mapping decision record.

Field	Purpose
decision_id	Unique identifier for the mapping or review decision.
source_record_id	Identifier of the source evidence being mapped.
candidate_asset_ids	Candidate canonical contexts considered.
method_tier	Deterministic, similarity, machine-assisted recommendation, or human review.
rule_or_model_version	Versioned logic that produced the route or recommendation.
score_or_confidence	Reproducible score or stated uncertainty where applicable.
threshold_configuration	Named configuration in force at the decision time.
routing_outcome	Accepted, rejected, review required, deferred, escalated, unresolved, or contradiction.
reviewer_or_actor	Authorized person or system process responsible for the disposition.
reason_code	Controlled explanation for the outcome.
evidence_references	Source records and time-validity information supporting the decision.
decision_timestamp	Time of decision.
effective_interval	Period during which the relationship is considered valid, if applicable.
model_profile_version	Canonical model profile in force when the decision was produced.
schema_version	Schema version of the persisted decision record.

Appendix C. Glossary

Asset evidence. A source record or controlled reference that supports, challenges, or contextualizes a representation of an asset.

Blocking. The restriction of candidate generation to plausible comparison sets before scoring.

Canonical asset. A governed representation of an asset context, distinct from the source records that describe it.

CIM. The IEC Common Information Model defined across standards including IEC 61970 and IEC 61968.

Contradiction. A typed disagreement between evidence classes, identifiers, statuses, or time contexts.

Derived status. An operator-governed information product produced under a documented rule from a defined evidence set.

Evidence record. A preserved source record, document reference, event, observation, or transaction.

Functional location. The role or position occupied in the network or operating hierarchy.

Physical equipment identity. The identity of the installed physical item, commonly associated with serial or manufacturer data.

Provenance. Information that explains the origin, transformation, decision path, and responsibility associated with a result.

Reconciliation. The controlled process of evaluating whether source records refer to the same asset context.

Telemetry identity. The identity of observation channels such as tags or measurement points.

Unknown. A safe output indicating that available evidence does not support another approved status.